

**Ex.No: 7**

## **Exercises On Hooks**

**Date:**

**Aim:** To implement the react exercises on hooks

**Program:**

### **7.a) useState**

- 1. You are developing a quiz app where the user's selected answer must be stored for each question. How would you use useState to track the chosen options?*

Solution:

```
import React, { useState } from "react";
```

```
export default function QuizApp() {
```

```
  const questions = [
```

```
    {
```

```
      id: 1,
```

```
      question: "What is 2 + 2?",
```

```
      options: ["3", "4"],
```

```
      correct: "4",
```

```
    },
```

```
    {
```

```
      id: 2,
```

```
      question: "Capital of France?",
```

```
      options: ["London", "Paris"],
```

```
      correct: "Paris",
```

```
    },
```

```
    {
```

```
      id: 3,
```

```
      question: "Sun rises in the?",
```

```
    options: ["East", "West", "South", "North"],
    correct: "East",
  }
];
```

```
const [answers, setAnswers] = useState({});
const [submitted, setSubmitted] = useState(false);
```

```
const handleAnswer = (id, option) => {
  setAnswers((prev) => ({
    ...prev,
    [id]: option,
  }));
};
```

```
const handleSubmit = () => {
  setSubmitted(true);
};
```

```
const score = Object.keys(answers).filter(
  (id) =>
    answers[id] ===
    questions.find((q) => q.id.toString() === id.toString()).correct
).length;
```

```
return (
  <div style={{ maxWidth: "500px", margin: "20px auto", fontFamily: "Arial"
}}>
    <h1>Quiz App</h1>
    {!submitted ? (
```

</>

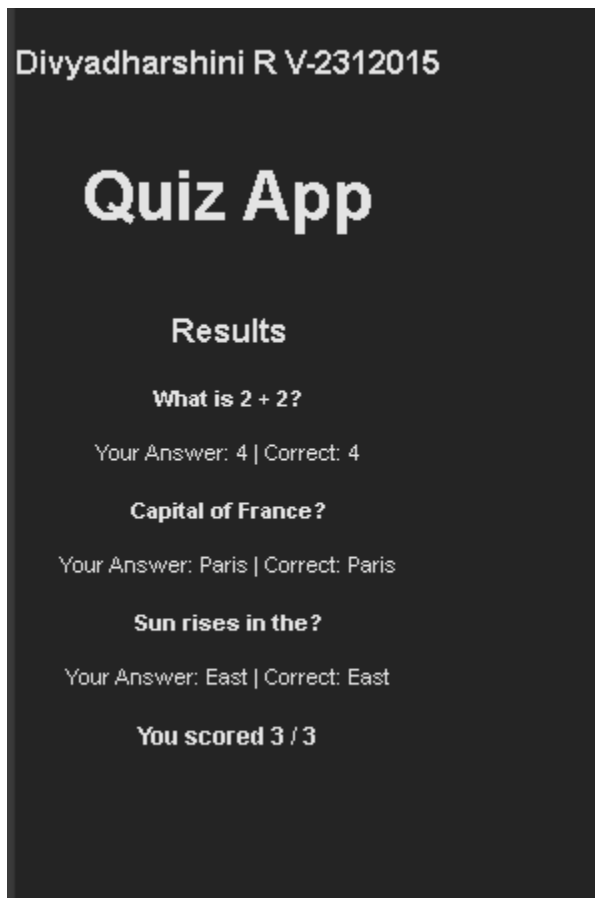
```
{questions.map((q) => (  
  <div key={q.id} style={{ marginBottom: "20px" }}>  
    <p>{q.question}</p>  
    {q.options.map((opt) => (  
      <label key={opt} style={{ display: "block", marginBottom: "5px" }}>  
        <input  
          type="radio"  
          name={`q${q.id}`}  
          value={opt}  
          checked={answers[q.id] === opt}  
          onChange={() => handleAnswer(q.id, opt)}  
        />  
        {opt}  
      </label>  
    ))}  
  </div>  
))}  
  <button onClick={handleSubmit}>Submit</button>  
</>  
) : (  
  <div>  
    <h2>Results</h2>  
    {questions.map((q) => (  
      <div key={q.id} style={{ marginBottom: "20px" }}>  
        <p>  
          <strong>{q.question}</strong>  
        </p>  
        <p>  
          Your Answer: {answers[q.id] || "Not Answered"} | Correct: {" "}
```

```

        {q.correct}
      </p>
    </div>
  )))}
  <h3>
    You scored {score} / {questions.length}
  </h3>
</div>
  )}
</div>
);
}

```

### Output:



2. *In a shopping app, you want to maintain the list of items added to the cart. How can useState help manage this dynamic array?*

**Solution:**

```
import React, { useState } from "react";

export default function Cart() {
  const [cart, setCart] = useState([]);

  const products = [
    { id: 1, name: "T-Shirt", price: 499 },
    { id: 2, name: "Jeans", price: 999 },
    { id: 3, name: "Sneakers", price: 1999 },
  ];

  const addToCart = (product) => {
    setCart([...cart, product]);
  };

  const removeFromCart = (id) => {
    setCart(cart.filter((item, index) => index !== id));
  };

  const total = cart.reduce((sum, item) => sum + item.price, 0);

  return (
    <div className="min-h-screen bg-gray-50 flex flex-col items-center p-6">
      <h1 className="text-2xl font-bold mb-6">🛒 Shopping Cart</h1>

      { /* Product List */ }
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4 w-full max-w-4xl mb-10">
        {products.map((p) => (
          <div
            key={p.id}
            className="p-4 bg-white rounded-2xl shadow flex flex-col items-center"
          >
            <div className="text-lg font-semibold">{p.name}</div>
            <div className="text-gray-600 mb-3">₹{p.price}</div>
            <button
              onClick={() => addToCart(p)}
              className="px-4 py-2 rounded-xl bg-blue-500 text-white hover:bg-blue-600"
            >
              Add to Cart
            </button>
          </div>
        ))}
      </div>

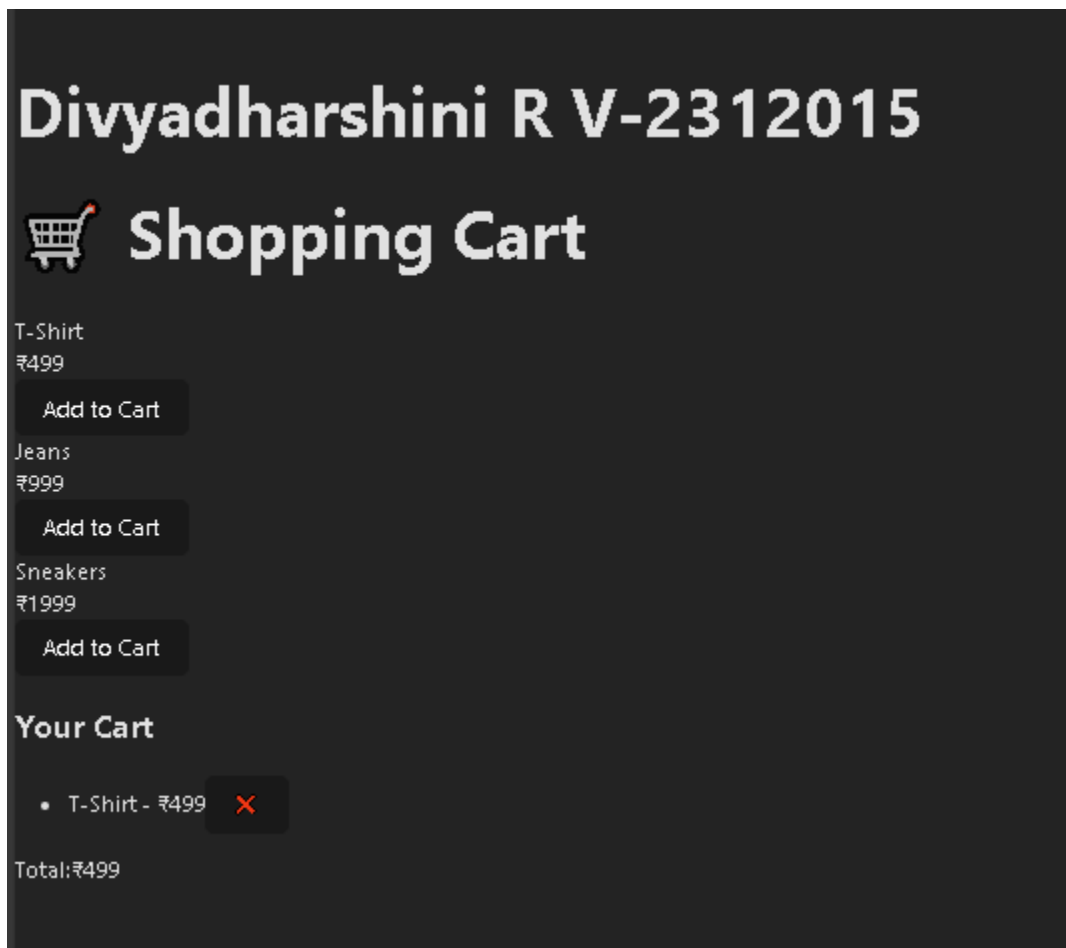
      { /* Cart Section */ }
```

```

<div className="w-full max-w-2xl bg-white shadow rounded-2xl p-6">
  <h2 className="text-xl font-semibold mb-4">Your Cart</h2>
  {cart.length === 0 ? (
    <p className="text-gray-500">Cart is empty</p>
  ) : (
    <ul className="space-y-3">
      {cart.map((item, index) => (
        <li
          key={index}
          className="flex justify-between items-center border-b pb-2"
        >
          <span>
            {item.name} - ₹{item.price}
          </span>
          <button
            onClick={() => removeFromCart(index)}
            className="text-red-500 hover:text-red-700"
          >
            ✖
          </button>
        </li>
      ))}
    </ul>
  )}
  {cart.length > 0 && (
    <div className="flex justify-between mt-4 text-lg font-bold">
      <span>Total:</span>
      <span>₹{total}</span>
    </div>
  )}
</div>
</div>
);
}

```

## Output:



3. You are building a toggle button for showing/hiding a modal popup. How would you use `useState` to track the modal's visibility state?

```
import React, { useState } from "react";

export default function ModalToggle() {
  const [isOpen, setIsOpen] = useState(false);

  const modalStyle = {
    display: isOpen ? "flex" : "none",
    position: "fixed",
    top: 0,
    left: 0,
    width: "100%",
    height: "100%",
    backgroundColor: "rgba(0,0,0,0.5)",
    alignItems: "center",
    justifyContent: "center",
  };

  const modalContentStyle = {
    background: "#fff",
    padding: "25px",
  };
}
```

```

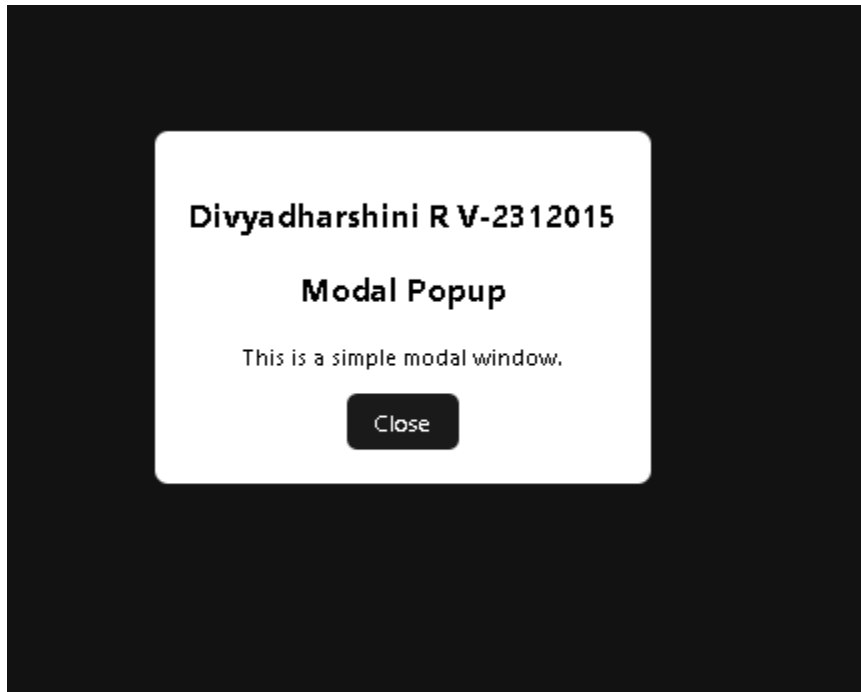
borderRadius: "10px",
minWidth: "300px",
textAlign: "center",
};

return (
  <div>
    <button onClick={() => setIsOpen(true)}>Open Modal</button>

    <div style={modalStyle}>
      <div style={modalContentStyle}>
        <h2>Modal Popup</h2>
        <p>This is a simple modal window.</p>
        <button onClick={() => setIsOpen(false)}>Close</button>
      </div>
    </div>
  </div>
);
}

```

OUTPUT:



## 7b) useEffect

1. *In a weather app, you need to fetch live weather data from an API whenever the page loads. How can you use useEffect to achieve this?*



## Solution:

```
import React, { useEffect, useState } from "react";

function WeatherApp() {
  const [city, setCity] = useState("London");
  const [weather, setWeather] = useState(null);

  const fetchWeather = async (cityName) => {
    try {
      const response = await fetch(
`https://api.openweathermap.org/data/2.5/weather?q=${cityName}&units=metric&appid=82d142fdd0ec3952f54ce1c8e9dc2d7`
      );
      const data = await response.json();
      setWeather(data);
    } catch (error) {
      console.error("Error fetching weather:", error);
    }
  };

  useEffect(() => {
    fetchWeather(city);
  }, [city]);

  const handleSearch = (e) => {
    e.preventDefault();
    fetchWeather(city);
  };

  return (

    <div>
      <h1>Weather App</h1>

      <form onSubmit={handleSearch}>
        <input
          type="text"
          placeholder="Enter city name"
          value={city}
          onChange={(e) => setCity(e.target.value)}
        />

      </form>
    </div>
  );
}
```

```
{weather && weather.main ? (  
  <div>  
    <h2>{weather.name}</h2>  
    <p>Temperature: {weather.main.temp}°C</p>  
    <p>Weather: {weather.weather[0].description}</p>  
  </div>  
  ) : (  
    <p>No data available</p>  
  )}  
</div>  
);  
}  
  
export default WeatherApp;
```

## Output:

# Weather App

**Chennai**

Temperature: 33.25°C

Weather: overcast clouds

2. *You are tracking a user's scroll position to show a "Back to Top" button. How would you use `useEffect` to add and clean up the scroll event listener?*

```
import React, { useState, useEffect } from "react";

export default function BackToTop() {
  const [isVisible, setIsVisible] = useState(false);

  useEffect(() => {
    const handleScroll = () => {
      if (window.scrollY > 200) {
        setIsVisible(true);
      } else {
        setIsVisible(false);
      }
    };

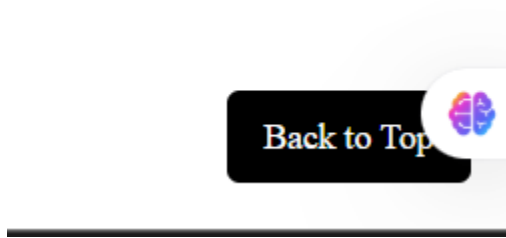
    window.addEventListener("scroll", handleScroll);

    // cleanup when component unmounts
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  const scrollToTop = () => {
    window.scrollTo({ top: 0, behavior: "smooth" });
  };

  return (
    <div style={{ height: "1500px", padding: "20px" }}>
      <h2>Scroll Down to See the Back to Top Button</h2>
      {isVisible && (
        <button
          onClick={scrollToTop}
          style={{
            position: "fixed",
            bottom: "20px",
            right: "20px",
            padding: "10px 15px",
            fontSize: "14px",
            background: "black",
            color: "white",
            border: "none",
            borderRadius: "5px",
            cursor: "pointer",
          }}
        >
          Back to Top
        </button>
      )}
    </div>
  );
}
```

```
</div>  
);  
}
```



*3, In a product details page, you need to fetch product info from the backend whenever the productId changes. How would you use useEffect to refetch data based on this dependency?*

```
import { useEffect, useState } from "react";  
  
function ProductDetails({ productId = 2 }) {  
  const [product, setProduct] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);  
  
  useEffect(() => {  
    if (!productId) return;  
  
    const fetchProduct = async () => {  
      try {  
        setLoading(true);  
        setError(null);  
  
        console.log("Fetching product:", productId);  
  
        const response = await fetch(`https://fakestoreapi.com/products/${productId}`);  
        if (!response.ok) {  
          throw new Error("Failed to fetch product");  
        }  
  
        const data = await response.json();  
        console.log("Data received:", data);  
        setProduct(data);  
      } catch (err) {  
        setError(err.message);  
      } finally {  
        setLoading(false);  
      }  
    }  
  });  
}
```

```

    }
  };

  fetchProduct();
}, [productId]);

if (loading) return <p>Loading product...</p>;
if (error) return <p>Error: {error}</p>;
if (!product) return <p>No product found</p>;

return (
  <div>
    <ul style={{ listStyleType: "none", padding: 0 }}>
      <li><strong>Name:</strong> {product.title}</li>
      <li><img src={product.image} alt={product.title} width={200} /></li>
      <li><strong>Price:</strong> ₹{product.price}</li>
      <li><strong>Description:</strong> {product.description}</li>
    </ul>

    </div>
  );
}

export default ProductDetails;

```

## Output:

**Name:** Pierced Owl Rose Gold Plated Stainless Steel Double



**Price:** ₹10.99

**Description:** Rose Gold Plated Double Flared Tunnel Plug Earrings. Made of 316L Stainless Steel

## 7c)useContext

1. In a dashboard app, you need to share the logged-in user's profile information across multiple nested components. How can useContext help simplify this instead of prop drilling?

```
import React, { createContext, useContext, useState } from "react";
```

//① Create a Context

```
const UserContext = createContext();
```

//② Context Provider Component

```
function UserProvider({ children }) {  
  const [user] = useState({  
    name: "Jamuna Rani",  
    role: "Admin",  
    email: "jamuna@example.com"  
  });  
  
  return (  
    <UserContext.Provider value={user}>  
      {children}  
    </UserContext.Provider>  
  );  
}
```

//③ A nested component that consumes context

```
function UserProfile() {  
  const user = useContext(UserContext); // no props needed 🎉  
  return (  
    <div>  
      <h2>Welcome, {user.name} 🙌 </h2>  
      <p>Role: {user.role}</p>  
      <p>Email: {user.email}</p>  
    </div>  
  );  
}
```

//④ App Component

```
function DashboardApp() {  
  return (  
    <UserProvider>  
      <div>  
        <h1>Dashboard</h1>  
        <UserProfile />  
      </div>  
    </UserProvider>  
  );  
}
```

```
export default DashboardApp;
```

# Dashboard

Welcome, Divyadharshini 🙌

Role: Admin

Email: divya@example.com

2. A shopping cart has to display the total number of items in the header, sidebar, and footer simultaneously. How would you use useContext to manage this shared state?

```
import React, { createContext, useContext, useState } from "react";
```

```
//① Create Cart Context
```

```
const CartContext = createContext();
```

```
//② Cart Provider to wrap the app
```

```
function CartProvider({ children }) {  
  const [cartItems, setCartItems] = useState([]);
```

```
  // Add item function
```

```
  const addToCart = (item) => {  
    setCartItems((prev) => [...prev, item]);  
  };
```

```
  return (
```

```
    <CartContext.Provider value={{ cartItems, addToCart }}>  
      {children}  
    </CartContext.Provider>
```

```
  );
```

```
}
```

```
//③ Header Component
```

```
function Header() {
```

```
  const { cartItems } = useContext(CartContext);  
  return <h2>🛒 Items in Cart: {cartItems.length}</h2>;  
}
```

```
//④ Sidebar Component
```

```

function Sidebar() {
  const { cartItems } = useContext(CartContext);
  return <p>Sidebar → {cartItems.length} items</p>;
}

//5 Footer Component
function Footer() {
  const { cartItems } = useContext(CartContext);
  return <p>Footer → {cartItems.length} items</p>;
}

//6 Product Component (to add items)
function Product({ name }) {
  const { addToCart } = useContext(CartContext);
  return <button onClick={() => addToCart(name)}>Add {name}</button>;
}

//7 Main App
function ShoppingCartApp() {
  return (
    <CartProvider>
      <Header />
      <Sidebar />
      <Product name="T-shirt" />
      <Product name="Jeans" />
      <Footer />
    </CartProvider>
  );
}

export default ShoppingCartApp;

```



**Items in Cart: 6**

Sidebar → 6 items

Add T-shirt

Add Jeans

Footer → 6 items



## 7d)useCallback

1. You have a list of users, and each user has a “Follow” button. The click handler is defined inside the parent component, causing performance issues. How would you optimize it using useCallback?

```
import { useState } from "react";

function UserList() {
  const users = [
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" },
    { id: 3, name: "Charlie" },
  ];

  // Keep track of following users (by their IDs)
  const [following, setFollowing] = useState([]);

  const toggleFollow = (id) => {
    setFollowing((prev) =>
      prev.includes(id)
        ? prev.filter((userId) => userId !== id) // unfollow
        : [...prev, id] // follow
    );
  };

  return (
    <div>
      <ul>
        {users.map((user) => {
          const isFollowing = following.includes(user.id);
          return (
            <li key={user.id}>
              {user.name}{""}
              <button
                onClick={() => toggleFollow(user.id)}
                style={{
                  background: isFollowing ? "lightgreen" : "lightcoral",
                  border: "none",
                  borderRadius: "8px",
                  padding: "5px 10px",
                  cursor: "pointer",
                }}
              >
                {isFollowing ? "Following" : "Follow"}
              </button>
            </li>
          );
        })}
      </ul>
    </div>
  );
}
```

```

    }}
  </ul>

  <p>
    Following: {following.length > 0 ? following.join(", ") : "None"}
  </p>
</div>
);
}

export default UserList;

```

- Alice Following
- Bob Following
- Charlie Follow

Following: 1, 2

2. You are working on a form with multiple fields. Each input has an onChange handler passed from the parent. How can useCallback help reduce unnecessary re-renders of these input components?

```

import { useState, useCallback } from "react";

function InputField({ label, value, onChange }) {
  console.log(`${label} rendered`); // To check re-renders

  return (
    <div>
      <label>{label}: </label>
      <input value={value} onChange={onChange} />
    </div>
  );
}

function Form() {
  const [formData, setFormData] = useState({

```

```
    name: "",
    email: "",
    password: ""
  });
```

```
// ✅ useCallback ensures stable function reference
const handleChange = useCallback(
  (field) => (e) => {
    setFormData((prev) => ({ ...prev, [field]: e.target.value }));
  },
  [] // no dependencies → stable across re-renders
);
```

```
return (
  <div>
    <InputField
      label="Name"
      value={formData.name}
      onChange={handleChange("name")}
    />
    <InputField
      label="Email"
      value={formData.email}
      onChange={handleChange("email")}
    />
    <InputField
      label="Password"
      value={formData.password}
      onChange={handleChange("password")}
    />
  </div>
);
}
```

```
export default Form;
```

Name:

Email:

Password:

3. iii) In a to-do app, the child component renders a list of tasks and receives a `deleteTask` function from the parent. How can you ensure this function reference stays stable across renders using `useCallback`?

Program:

```
import { useState, useCallback } from "react";

function TaskList({ tasks, onDelete })
{ console.log("Rendering TaskList");
  return (
    <ul>
      {tasks.map((task) => (
        <li key={task.id}>
          {task.text}
          <button onClick={() => onDelete(task.id)}>Delete</button>
        </li>
      ))}
    </ul>
  );
}

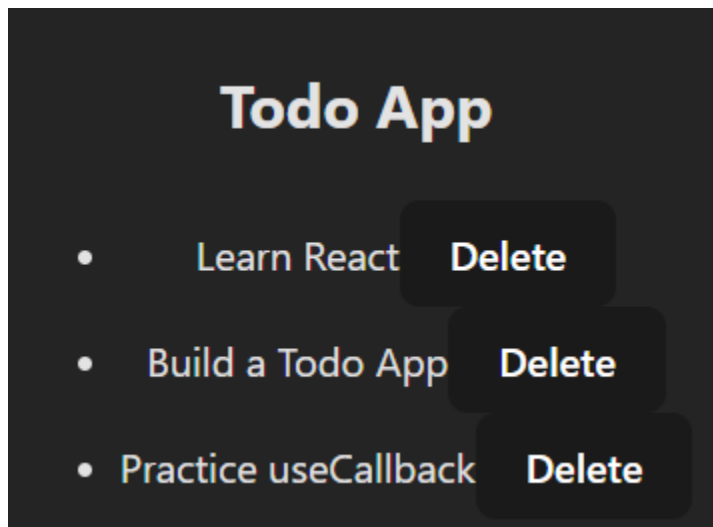
export default function Task() {
```

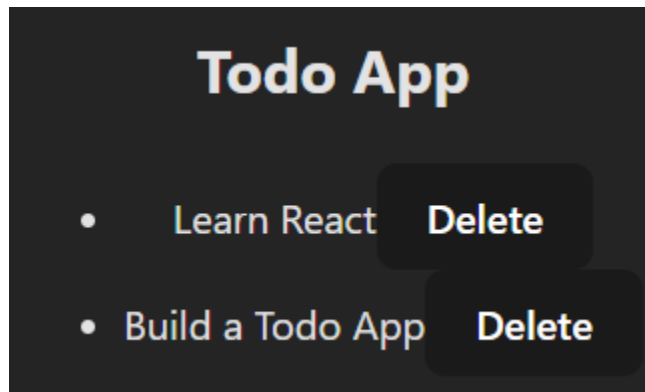
```
const [tasks, setTasks] = useState([
  { id: 1, text: "Learn React" },
  { id: 2, text: "Build a Todo App" },
  { id: 3, text: "Practice useCallback" }
]);

const deleteTask = useCallback((id) => {
  setTasks((prev) => prev.filter((task) => task.id !== id));
}, []);

return (
  <div>
    <h2>Todo App</h2>
    <TaskList tasks={tasks} onDelete={deleteTask} />
  </div>
);
}
```

Output:





## 7.e useMemo

1. In a financial dashboard, calculating the total revenue requires iterating over thousands of records. How can you prevent recalculating it on every render using useMemo?

Program:

```
// src/Dashboard.jsx
import React, { useState, useMemo } from "react";

// Sample transaction generator
const generateTransactions = (num) => { const transactions = [];
for (let i = 1; i <= num; i++) { transactions.push({
id: i,
description: `Transaction ${i}`,
amount: Math.floor(Math.random() * 1000) + 1, // Random amount 1-1000
});
}
return transactions;
};

function Dashboards() {
const [transactions, setTransactions] = useState(generateTransactions(5000));

// Memoize total revenue
const totalRevenue = useMemo(() => { console.log("Calculating total revenue...");
return transactions.reduce((sum, t) => sum + t.amount, 0);
}, [transactions]);

const addTransaction = () => { const newTransaction = {
id: transactions.length + 1,
description: `Transaction ${transactions.length + 1}`, amount: Math.floor(Math.random() * 1000) + 1,
};
setTransactions([...transactions, newTransaction]);
};
```

```
return (  
  <div style={{ padding: "20px", fontFamily: "Arial" }}>  
    <h1>Financial Dashboard</h1>  
    <h2>Total Revenue: ${totalRevenue}</h2>  
  
    <button onClick={addTransaction} style={{ marginBottom: "20px" }}> Add Transaction  
    </button>  
  
    <table border="1" cellPadding="5" cellSpacing="0">  
  
      <thead>  
        <tr>  
          <th>ID</th>  
          <th>Description</th>  
          <th>Amount ($)</th>  
        </tr>  
      </thead>  
      <tbody>  
        {transactions.slice(0, 20).map((t) => (  
          <tr key={t.id}>  
            <td>{t.id}</td>  
            <td>{t.description}</td>  
            <td>{t.amount}</td>  
          </tr>  
        ))}  
      </tbody>  
    </table>  
    <p>Showing first 20 transactions out of {transactions.length}</p>  
  </div>  
);  
}  
  
export default Dashboards;
```

Output:

# Financial Dashboard

Total Revenue: \$2482417

Add Transaction

| ID | Description    | Amount (\$) |
|----|----------------|-------------|
| 1  | Transaction 1  | 177         |
| 2  | Transaction 2  | 87          |
| 3  | Transaction 3  | 331         |
| 4  | Transaction 4  | 184         |
| 5  | Transaction 5  | 755         |
| 6  | Transaction 6  | 183         |
| 7  | Transaction 7  | 754         |
| 8  | Transaction 8  | 275         |
| 9  | Transaction 9  | 996         |
| 10 | Transaction 10 | 666         |
| 11 | Transaction 11 | 831         |
| 12 | Transaction 12 | 159         |
| 13 | Transaction 13 | 7           |

|    |                |     |
|----|----------------|-----|
| 13 | Transaction 13 | 7   |
| 14 | Transaction 14 | 228 |
| 15 | Transaction 15 | 201 |
| 16 | Transaction 16 | 28  |
| 17 | Transaction 17 | 208 |
| 18 | Transaction 18 | 106 |
| 19 | Transaction 19 | 158 |
| 20 | Transaction 20 | 146 |

Showing first 20 transactions out of 5000



2. You are building a search bar that filters a large dataset of products. The filtering logic is slow. How can you optimize this so filtering only happens when the search term changes?

Program:

```
import React, { useState, useMemo } from "react";

function ProductSearch({ products }) {
  const [searchTerm, setSearchTerm] = useState("");

  const filteredProducts = useMemo(() => {
    console.log("Filtering products..."); // shows when recalculated
    return products.filter((product) =>
      product.name.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }, [searchTerm, products]);

  return (
    <div>
      <input
        type="text"
        placeholder="Search products..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        style={{ marginBottom: "10px", padding: "5px", width: "250px" }}
      />
    </div>
  );
}
```

```
    <ul>
      {filteredProducts.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  </div>
);
}
```

```
export default ProductSearch;
```

```
// src/App.jsx
```

```
import React from "react";
```

```
import ProductSearch from "../usememo/search";
```

```
function App() {
```

```
// Example dataset of products (can be thousands in real apps)
```

```
const products = [
```

```
{ id: 1, name: "Apple iPhone 15" },
```

```
{ id: 2, name: "Samsung Galaxy S23" },
```

```
{ id: 3, name: "Google Pixel 8" },
```

```
{ id: 4, name: "OnePlus 11" },
```

```
{ id: 5, name: "Sony Xperia 5" },
```

```
// ... imagine thousands more
```

```
];
```

```

return (
  <div style={{ padding: "20px", fontFamily: "Arial" }}>
    <h1>Product Search</h1>

    <ProductSearch products={products}/>
  </div>
);
}
export default App;

```

Output:

## Product Search

- Apple iPhone 15
- Samsung Galaxy S23
- Google Pixel 8
- OnePlus 11
- Sony Xperia 5

7)f) i) **Track Previous State** : Create a counter (`count`) that increases on button click. Use `useRef` to store the **previous value** of `count`. Display both the current and previous values.

Program:

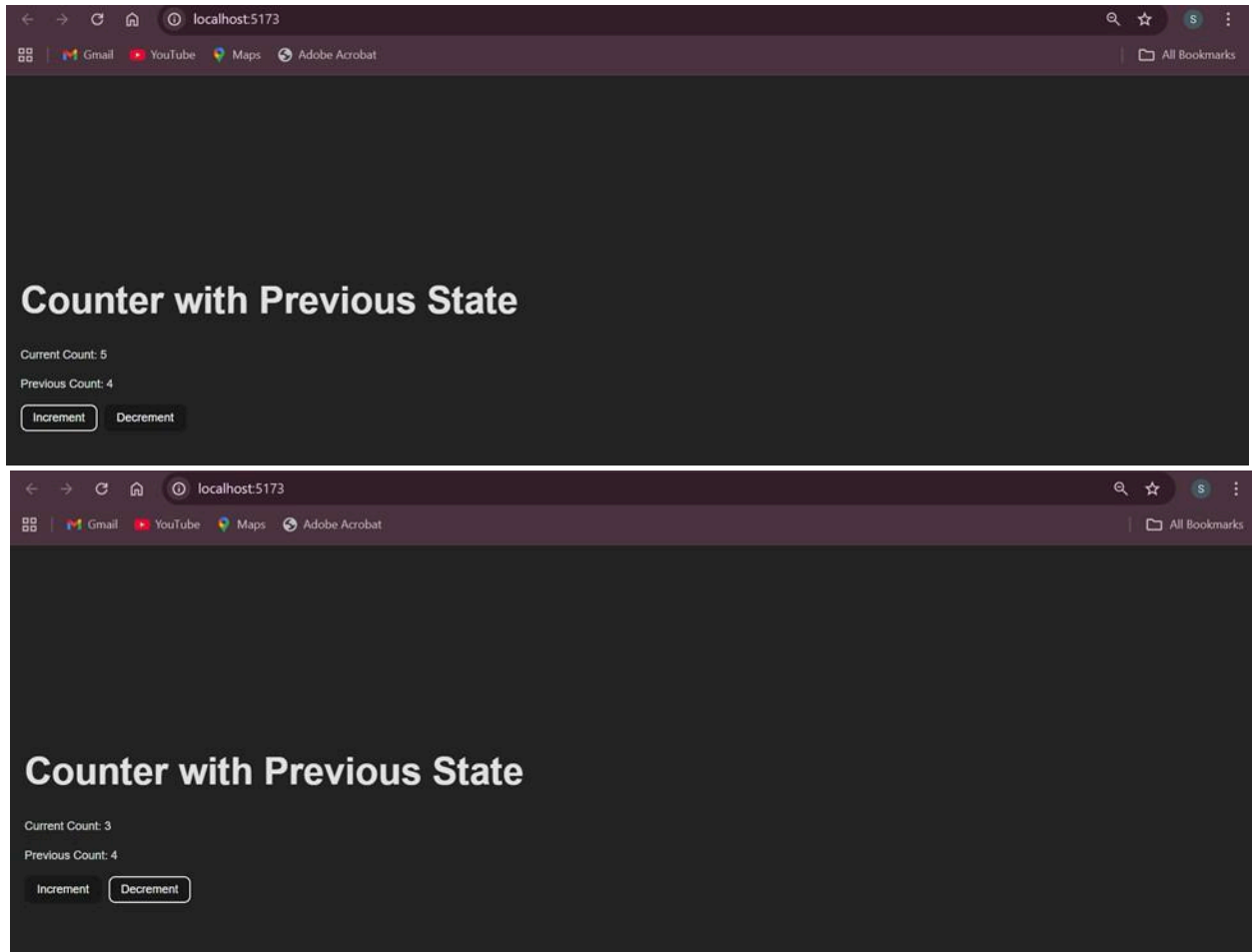
```
// src/Counter.jsx
```

```
import React, { useState, useRef, useEffect } from "react";
```

```
function Counter() {  
  // 1. State for current count  
  const [count, setCount] = useState(0);  
  
  // 2. Ref to store previous count  
  const prevCountRef = useRef();  
  
  // 3. Update previous count after every render  
  useEffect(() => {  
    prevCountRef.current = count;  
  }, [count]);  
  
  return (  
    <div style={{ padding: "20px", fontFamily: "Arial" }}>  
      <h1>Counter with Previous State</h1>  
  
      {/* 4. Display current and previous values */}  
      <p>Current Count: {count}</p>  
      <p>Previous Count: {prevCountRef.current ?? "N/A"}</p>  
  
      {/* 5. Buttons to increment & decrement */}  
      <button  
        onClick={() => setCount(count + 1)}  
        style={{ padding: "8px 16px", marginRight: "10px" }}  
      >  
        Increment  
      </button>  
  
      <button  
        onClick={() => setCount(count - 1)}
```

```
        style={{ padding: "8px 16px" }}
      >

        Decrement
      </button>
    </div>
  );
}
export default Counter;
Output:
```



7)f) ii) Scroll to an Element: Create a long list of items. Use useRef to point to a specific item (e.g., the 50th item). Add a button that scrolls directly to that item.

Program:

```
// src/ScrollList.jsx

import React, { useRef } from "react";

function ScrollList() {

  // Ref for the 50th item

  const item50Ref = useRef(null);

  // Scroll handler

  const scrollToItem = () => { item50Ref.current?.scrollIntoView({ behavior: "smooth",
  block: "center",
  });

  };

  return (

    <div style={{ padding: "20px" }}>

      <button onClick={scrollToItem}

        style={{ marginBottom: "20px", padding: "10px 20px" }}


```

Scroll to Item 50

```
</button>

  /* Scrollable container */

  <div style={{

    height: "400px", overflowY: "scroll", border: "1px solid #ccc", padding: "10px",

  }}


```

```
{Array.from({ length: 100 }, (_, index) => { const itemNumber = index + 1;
```

```

return (
  <div key={itemNumber}
    ref={itemNumber === 50 ? item50Ref : null} style={{
  }}

  Item {itemNumber}
  </div>
);
}})
</div>
</div>
);
}
export default ScrollList;

// src/App.jsx
import React from "react";
import ScrollList from "../ScrollList";

function App() { return (
  <div>

  <h1 style={{ textAlign: "center" }}>Scroll to an Element Example</h1>

  <ScrollList />

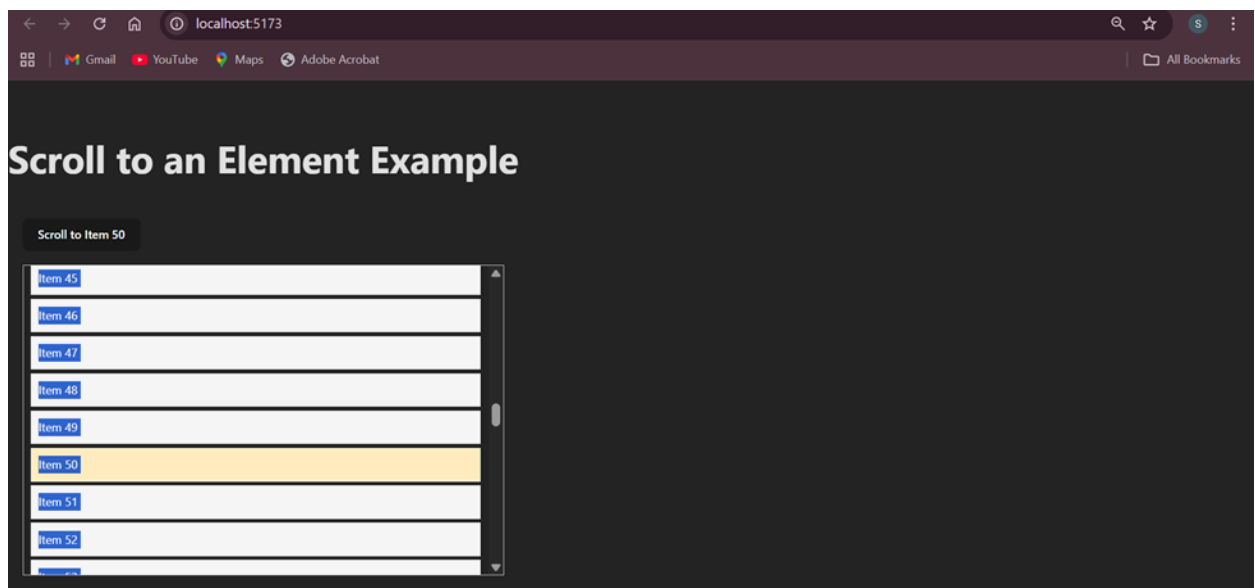
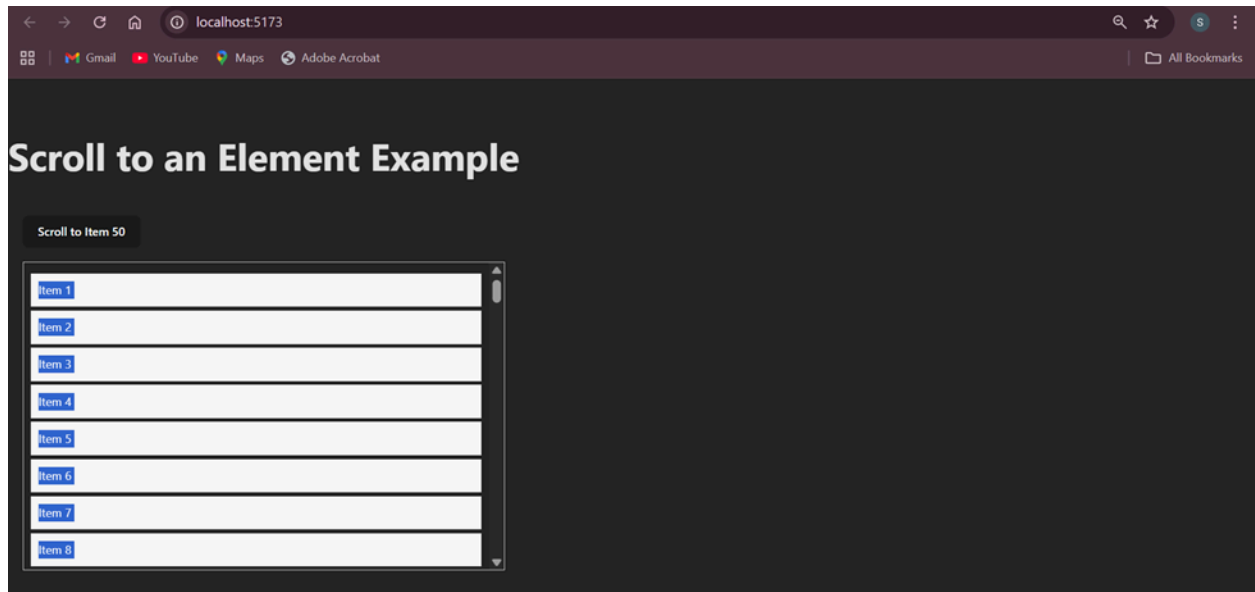
  </div>

);
}

export default App;

```

Output:



7)g) i) `useDebounce` Hook: Accept a `value` and `delay`. Return the debounced value (only update after no changes for given delay).

Program:

```
// src/hooks/useDebounce.js
import { useState, useEffect } from "react";
```



```

function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // Set a timer
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup if value changes before delay
    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}

export default useDebounce;

// src/SearchBar.jsx
import React, { useState, useMemo } from "react";
import useDebounce from "../hooks/useDebounce";

function SearchBar({ products }) {
  const [searchTerm, setSearchTerm] = useState("");
  const debouncedSearchTerm = useDebounce(searchTerm, 500); // 500ms delay

```

```

// Filter only when debouncedSearchTerm updates
const filteredProducts = useMemo(() => {
  console.log("Filtering...");
  return products.filter((p) =>
    p.name.toLowerCase().includes(debouncedSearchTerm.toLowerCase())
  );
}, [debouncedSearchTerm, products]);

return (
  <div style={{ padding: "20px" }}>
    <input
      type="text"
      placeholder="Search products..."
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      style={{ padding: "8px", width: "250px", marginBottom: "15px" }}
    />

    <ul>
      {filteredProducts.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  </div>
);
}

export default SearchBar;

```

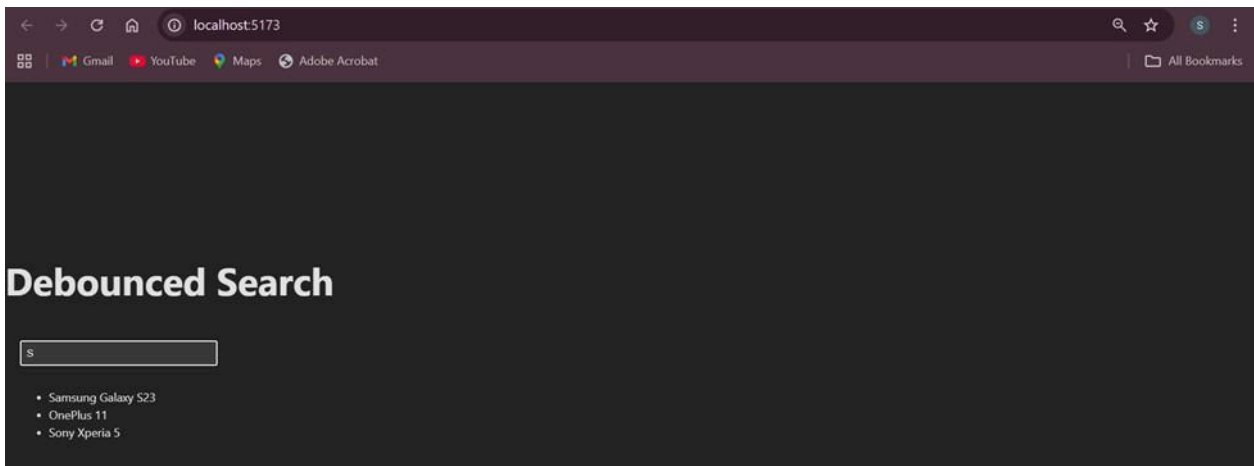
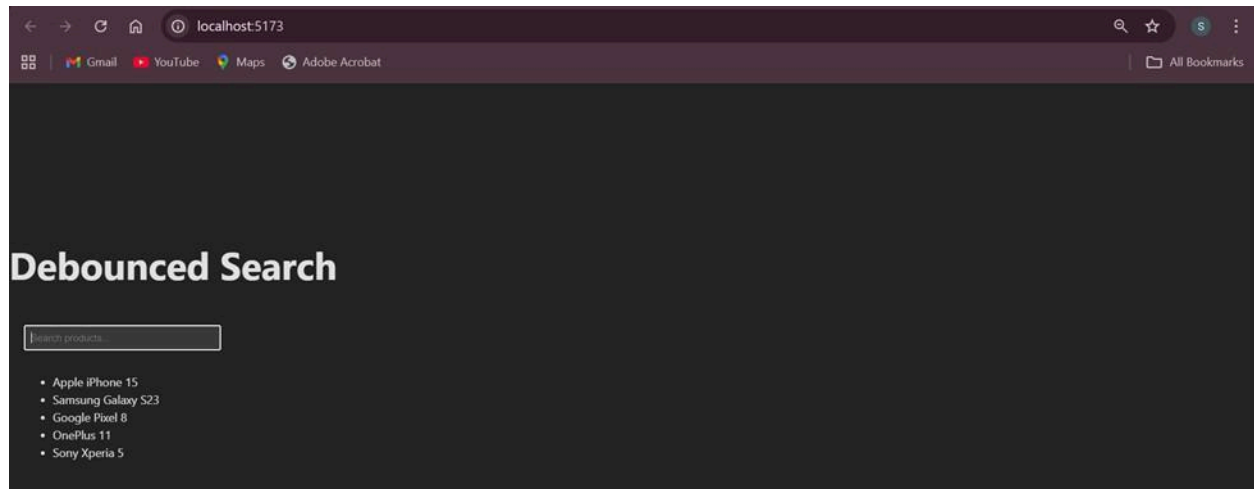
```
// src/App.jsx
import React from "react";
import SearchBar from "../SearchBar";

function App() {
  const products = [
    { id: 1, name: "Apple iPhone 15" },
    { id: 2, name: "Samsung Galaxy S23" },
    { id: 3, name: "Google Pixel 8" },
    { id: 4, name: "OnePlus 11" },
    { id: 5, name: "Sony Xperia 5" },
    // Imagine thousands more...
  ];

  return (
    <div>
      <h1 style={{ textAlign: "center" }}>Debounced Search</h1>
      <SearchBar products={products} />
    </div>
  );
}

export default App;
```

Output:



7)g) ii) `useEventListener` Hook: Create a hook that attaches/detaches an event listener to any target (`window`, `document`, `ref.current`).

Program:

```
// src/useEventListener.js
```

```
import { useEffect, useRef } from "react";
```

```
function useEventListener(eventType, handler, element = window) {  
  const savedHandler = useRef();
```

```
  // Update ref if handler changes
```

```
  useEffect(() => {
```

```
savedHandler.current = handler;
```

```
}, [handler]);
```

```
useEffect(() => {
```

```
  const targetElement = element && element.addEventListener ? element : null;
```

```
  if (!targetElement) return;
```

```
  // Event listener that calls the latest handler from ref
```

```
  const eventListener = (event) => savedHandler.current(event);
```

```
  targetElement.addEventListener(eventType, eventListener);
```

```
  // Cleanup
```

```
  return () => {
```

```
    targetElement.removeEventListener(eventType, eventListener);
```

```
  };
```

```
}, [eventType, element]);
```

```
}
```

```
export default useEventListener;
```

```
// src/components/WindowResizeLogger.jsx
```

```
import React, { useState } from "react";
```

```
import useEventListener from "../useEventListener";
```

```
function WindowResizeLogger() {
```

```
  const [size, setSize] = useState({
```

```
    width: window.innerWidth,
```

```
    height: window.innerHeight,
```

```
});
```

```
useEventListener("resize", () => {  
  setSize({ width: window.innerWidth, height: window.innerHeight });  
});
```

```
return (  
  <div>  
    <h2>Window Size</h2>  
    <p>Width: {size.width}</p>  
    <p>Height: {size.height}</p>  
  </div>  
);  
}
```

```
export default WindowResizeLogger;
```

```
// src/components/ClickBox.jsx  
import React, { useRef, useState, useEffect } from "react";  
import useEventListener from "../useEventListener";  
function ClickBox() {  
  const boxRef = useRef(null);  
  const [clicks, setClicks] = useState(0);  
  
  useEffect(() => {  
    if (!boxRef.current) return;  
    }, []);
```

```

useEventListener(
  "click",
  () => setClicks((c) => c + 1),
  boxRef.current // attach only to this element
);

return (
  <div>
    <div
      ref={boxRef}
      style={{
        width: "200px",
        height:
          "200px",
        background: "#90ee90",
        display: "flex",
        alignItems: "center",
        justifyContent:
          "center", cursor:
          "pointer",
      }}
    >
      Click Me
    </div>
    <p>Box Clicks: {clicks}</p>
  </div>
);
}

export default ClickBox;

```

```
// src/App.jsx
import React from "react";
import WindowResizeLogger from
"./WindowResizeLogger"; import ClickBox from
"./ClickBox";
function App() {
  return (
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h1>useEventListener Hook Demo</h1>
      <WindowResizeLogger />
      <hr />
      <ClickBox />
    </div>
  );
}
export default App;
```

Output:



Box Clicks: 8



**Rubrics:**

| Problem Analysis<br>(10) | Coding & Implementation<br>(10) | Time Management<br>(10) | Viva<br>(10) | Total<br>(40) |
|--------------------------|---------------------------------|-------------------------|--------------|---------------|
|                          |                                 |                         |              |               |

**Result:**

Thus the javascript programs on Hooks were successfully executed and verified the output